Environment Setup and Your First Playwright Test

COURSE SPECIFICATIONS

• Track: Playwright Automation Testing (JavaScript) • Level: Beginner
• Focus: Local Setup, CLI Wizard & First Test Script • Prerequisites: Day 2 JavaScript Essentials
• Tools: Node.js (Active LTS), npm, VS Code, Chromium • Test Runner: @playwright/test

Set up your local automation workstation with Node.js and npm, initialize a clean JavaScript Playwright project, write your first browser test, and execute it in both headless and headed modes.

Day 3 Learning Objectives

'Explain the purpose of Node.js, npm, VS Code, and Playwright in modern automation testing.
'Create a local Playwright project directory and install Playwright Test via the interactive CLI wizard.
'Understand the structure of a Playwright project including tests/, playwright.config.js, package.json, and node_modules/.
'Write a test in pure JavaScript using test, expect, and the page fixture to navigate and verify elements.
'Execute tests from your terminal in both headless and headed browser modes.
'Understand passing vs failing test outcomes, assertions, and how to safely troubleshoot setup errors.

Before installing any software, let us understand the four core tools you will use as an automation QA engineer every single day.

1. Node.js (The JavaScript Runtime)
• **What it is: Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside a web browser, directly on your computer's operating system.**
• **Why Playwright projects use Node.js: Browsers run JavaScript inside web pages. However, your test scripts live on your local machine and need to launch browsers, read configuration files, and access your terminal. Node.js makes that possible.**
• **Browser JS vs Node.js: In a browser, JavaScript has access to the `window` and `document` objects to interact with the webpage. In Node.js, JavaScript has access to file systems, network sockets, operating system processes, and terminal input/output.**
• **Node.js Version Requirement: Always use an Active LTS (Long Term Support) version of Node.js compatible with the current Playwright release. Check the official Playwright documentation (playwright.dev) for the latest minimum version requirements rather than hardcoding an outdated release number.**

2. npm (Node Package Manager)
• **What it is: npm is the default package manager distributed with Node.js. It is the world's largest software registry.**
• **Its role: npm downloads, installs, and manages external code libraries (packages) that your project depends on—such as `@playwright/test`.**
• **package.json: This is the manifest file located at the root of your project. It records project metadata, scripts, and lists the exact packages and versions your project needs.**

3. Visual Studio Code (VS Code)
• **What it is: A lightweight, free, and powerful code editor developed by Microsoft.**
• **Why testers love it: VS Code provides outstanding JavaScript syntax highlighting, autocompletion (IntelliSense), an integrated terminal, Git integration, and the official Playwright Test extension for one-click test execution and interactive debugging.**

4. The Terminal (Command Line Interface)
• **What it is: A text-based interface used to control your operating system by typing commands. (On macOS it is Terminal or iTerm2; on Windows it is PowerShell or Command Prompt).**
• **How we use it: You use the terminal to create project folders, install packages, run automated test suites, and view pass/fail logs.**

Core Automation Toolset Summary

Tool	Primary Purpose	Why Playwright Needs It
Node.js	Executes JavaScript on your computer	Runs test runner processes, parses config files, and communicates with browser engines
npm	Package manager and registry	Installs @playwright/test and downloads bundled browser binaries (Chromium, Firefox, WebKit)
VS Code	Code editor and workspace	Provides a friendly environment to write, edit, format, and organize JavaScript test scripts
Terminal	Command line prompt	Triggers test execution commands (npx playwright test) and displays live test results

Verifying Tool Installations (Commands to check if Node.js and npm are installed on your workstation.)

```
node --version
npm --version
```

Expected Output: v20.18.0
10.8.2

Now that you understand the tools, let us walk through setting up a clean Playwright project from scratch.

Prerequisites Before Starting
• A computer running macOS, Windows, or Linux.
• Node.js installed (LTS version).
• Visual Studio Code installed.
• An active internet connection (to download packages and browser binaries).

Step 1: Verify Node.js and npm
Open your terminal and run:
```bash
node --version
npm --version
```

• What this does: Queries your system to ensure Node and npm are recognized in your PATH environment variable.
• Successful output: You will see version numbers printed (e.g. `v20.x.x` or higher, and npm `10.x.x`). If you see `command not found`, install Node.js from nodejs.org first.

Step 2: Create a Dedicated Project Folder
Navigate to where you keep projects and create a new directory:
```bash
mkdir playwright-js-practice
cd playwright-js-practice
```

• `mkdir`: Creates a new folder named `playwright-js-practice`.
• `cd`: Changes your current working directory into the newly created folder. Everything we install will live inside this folder.

Step 3: Initialize the Project
Inside your new folder, generate your `package.json` file:
```bash
npm init -y
```

• What this does: The `-y` flag stands for "yes to all defaults". It creates a basic `package.json` file with default project settings in one second.

Step 4: Install Playwright Test via the Setup Wizard
Run the official Playwright initialization command:
```bash
npm init playwright@latest
```

The setup wizard will ask you a series of interactive prompts in your terminal:
1. **Do you want to use TypeScript or JavaScript?**
   Ø=ÜI **Choose JavaScript.** (Use your arrow keys to select JavaScript and press Enter).
2. **Where to put your end-to-end tests?**
   Ø=ÜI Press Enter to accept the default: **`tests`**.
3. **Add a GitHub Actions workflow?**
   Ø=ÜI Choose **false** (or `N`) for now; we will build CI/CD on Day 12.
4. **Install Playwright browsers (can be done manually)?**
   Ø=ÜI Choose **true** (or `Y`) so Playwright automatically downloads Chromium, Firefox, and WebKit binaries.

Safe Setup Note: If you run this command inside an existing project that already has files, follow the prompts carefully so you do not overwrite custom files. Never run destructive cleanup commands like deleting your configuration.

Step 5: Manual Browser Installation (If Skipped)
If you ever need to download or refresh the browser binaries later, run:

```bash
npx playwright install
```

Linux Users: On Linux distributions, you may also need to install operating-system level dependencies by running:
`npx playwright install --with-deps`

Project Initialization Sequence (The step-by-step terminal commands to create and initialize a project.)

```
mkdir playwright-js-practice
cd playwright-js-practice
npm init -y
npm init playwright@latest
```

Expected Output: ' Do you want to use TypeScript or JavaScript? · JavaScript
' Where to put your end-to-end tests? · tests
' Add a GitHub Actions workflow? · false
' Install Playwright browsers? · true

Once the setup wizard completes, open your project folder in VS Code (`code .`). You will see a clean, organized project structure.

Representative Project Directory Layout
Here is what a standard JavaScript Playwright project looks like:

```text
playwright-js-practice/
% % %  node_modules/
% % %  tests/
%    % % %  example.spec.js
% % %  package.json
% % %  package-lock.json
% % %  playwright.config.js
```

(Note: The exact files may vary slightly depending on your version and selected options, but these are the core components).

Detailed Breakdown of Each File and Folder

1. tests/ Folder
• **What it is: The dedicated home for your automated test files.**
• **How it works: By default, Playwright searches inside this directory for test files. You will create your test files here (such as `first-test.spec.js` or `login.spec.js`).**

2. example.spec.js
• **What it is: A sample test file provided by the Playwright team to demonstrate common actions and assertions.**
• **The .spec.js Convention: In modern testing, "spec" stands for specification. Naming your files with the `.spec.js` or `.test.js` extension tells the test runner that this file contains executable test scenarios.**

3. package.json
• **What it is: The project manifest file.**
• **What it contains:**
- Project name and version.
- Dependencies: shows `@playwright/test` listed under `devDependencies`.
- Scripts: custom shortcuts you can run with `npm test`.

4. package-lock.json
• **What it is: An automatically generated record of the exact versions of every library installed in `node_modules`.**
• **Why it matters: It ensures that every developer on your team and your CI/CD server install the exact identical package versions. You should never edit this file manually.**

5. playwright.config.js
• **What it is: The central control room for Playwright Test.**
• **What it configures:**
- Where test files are located (`testDir`).
- How long tests can run before timing out (`timeout`).
- Whether browsers run in headless or headed mode.
- Which browsers to test on (Chromium, Firefox, WebKit).
- Reporting formats (HTML report, list report).

6. node_modules/ Folder
• **What it is: The storage folder containing all the code downloaded from npm.**
• **Important rule: This folder is huge and should **never** be committed to Git. A `.gitignore` file should always exclude `node_modules/`. Anyone who clones your project can recreate it instantly by running `npm install`.**

Inspecting package.json Dependencies (A sample package.json showing Playwright installed as a devDependency.)

```
{
  "name": "playwright-js-practice",
  "version": "1.0.0",
  "scripts": {
    "test": "playwright test"
  },
  "devDependencies": {
    "@playwright/test": "^1.48.0"
  }
}
```

Now comes the exciting part: writing your first real automated browser test!

We will write a test that opens a browser, navigates to the public test website `https://example.com`, and verifies that the main heading *"Example Domain"* is visible on the screen.

(Public Site Note: Public websites can occasionally experience network outages. If a test fails unexpectedly, always verify that the site is accessible before assuming your test code is wrong!)

Create the Test File
Inside your project's `tests/` folder, create a new file named:
`tests/first-test.spec.js`

The CommonJS Test Example (Default for standard JS projects)
Add this code to `tests/first-test.spec.js`:

```javascript
const { test, expect } = require('@playwright/test');

test('homepage displays the expected heading', async ({ page }) => {
  // 1. Navigate to the website
  await page.goto('https://example.com');

  // 2. Assert that the heading is visible on screen
  await expect(
    page.getByRole('heading', { name: 'Example Domain' })
  ).toBeVisible();
});
```

Step-by-Step Code Explanation

1. `const { test, expect } = require('@playwright/test');`
• **`test`: The core function used to declare and structure test cases.**
• **`expect`: The assertion library used to verify actual conditions against expected outcomes.**
• **Note on Module Systems: If your project uses ES Modules (with `"type": "module"` in your `package.json`), you would use the `import` syntax instead:**
`import { test, expect } from '@playwright/test';`
Rule: Match your project's module system. Never mix require and import in the same file.

2. `test('homepage displays the expected heading', async ({ page }) => { ... });`
• The first argument is a clear, human-readable description of what this test verifies.
• The second argument is an `async` arrow function containing the test steps.
• **`{ page }` fixture: Playwright automatically creates a fresh, isolated browser page for this test and hands it to you as an argument. You do not have to manually launch or close the browser!**

3. `await page.goto('https://example.com');`
• Tells the browser to navigate to the specified URL.
• Notice the **`await` keyword: navigating across the web takes time. `await` tells JavaScript to pause until the web page finishes loading before moving to the next line.**

4. `page.getByRole('heading', { name: 'Example Domain' })`
• This is a **locator. It finds the element on the page with an accessible role of `heading` and the visible text `Example Domain`.**
• On Day 4, we will dive deeply into locators!

5. `await expect(...).toBeVisible();`
• This is a **Web-First Assertion.**
• It verifies that the heading is not only present in the HTML, but physically visible to users on the screen.
• Playwright automatically waits up to 5 seconds for the element to appear before failing, eliminating flaky timing issues.

First Playwright Test (CommonJS) (Complete example verifying page navigation and heading visibility on example.com.)

```
const { test, expect } = require('@playwright/test');

test('homepage displays the expected heading', async ({ page }) => {
  await page.goto('https://example.com');

  await expect(
    page.getByRole('heading', { name: 'Example Domain' })
  ).toBeVisible();
});
```

First Playwright Test (ES Module Syntax) (Equivalent test using ES module import syntax for projects with "type": "module".)

```
import { test, expect } from '@playwright/test';

test('homepage displays the expected heading', async ({ page }) => {
  await page.goto('https://example.com');

  await expect(
    page.getByRole('heading', { name: 'Example Domain' })
  ).toBeVisible();
});
```

Now that your test is written, it is time to run it from the terminal and understand what the test runner reports back.

1. Running the Full Test Suite (Headless Mode)
In your terminal, run:
```bash
npx playwright test
```

• By default, this runs all test files in your `tests/` directory.
• It runs in **Headless mode**—the browser executes silently in computer memory without opening a visible window. This is lightning fast!

2. Running a Specific Test File
If you have multiple files and only want to run your new test:
```bash
npx playwright test tests/first-test.spec.js
```

3. Running in Headed Mode (Watch the Browser Open!)
To visibly watch the browser open on your screen and see the page load in real time:
```bash
npx playwright test --headed
```

• A Chromium window will appear, navigate to `https://example.com`, check the heading, and close upon completion.

What Does the Terminal Output Mean?

Example: A Passing Test Run
```text
Running 3 tests using 3 workers

  ' 1 [chromium] › tests/first-test.spec.js:3:1 › homepage displays the expected heading (1.2s)
  ' 2 [firefox] › tests/first-test.spec.js:3:1 › homepage displays the expected heading (1.5s)
  ' 3 [webkit] › tests/first-test.spec.js:3:1 › homepage displays the expected heading (1.1s)
```

```
  3 passed (2.3s)
```

• **Workers:** Number of parallel threads running tests simultaneously.
• **Checkmark (` ' `):** The test completed all steps and all assertions passed.
• **Time (`1.2s`):** How many seconds the test took to run.

Example: A Failing Test Run
What happens if the heading text does not match? Let us say the test expected *"Wrong Heading Text"*:

```text
Running 1 test using 1 worker

  1) [chromium] › tests/first-test.spec.js:3:1 › homepage displays the expected heading % %

    Error: Timed out 5000ms waiting for expect(locator).toBeVisible()

    Locator: getByRole('heading', { name: 'Wrong Heading Text' })
    Expected: visible
    Received: <element(s) not found>
    Call log:
      - waiting for getByRole('heading', { name: 'Wrong Heading Text' }) to be visible

       7 |   await expect(
    >  8 |     page.getByRole('heading', { name: 'Wrong Heading Text' })
         |                          ^
       9 |   ).toBeVisible();

  1 failed
    [chromium] › tests/first-test.spec.js:3:1 › homepage displays the expected heading
```

How to Read the Failure Log:
1. **The Error Type: `Timed out 5000ms waiting for expect(locator).toBeVisible()`. Playwright waited 5 full seconds for the heading to show up, but it never appeared.**
2. **The Exact Line: Look at line 8 (`>`). Playwright pinpoints the exact line in your code where the test failed.**
3. **Assertion Failure vs Syntax Error:**
- **Assertion Failure: The code ran fine and the browser opened, but the application state did not match what we expected (e.g. heading was missing or wrong text).**
- **Syntax Error: Your JavaScript has a typo (like a missing parenthesis). The test fails immediately before any browser opens.**
- **Environment Error: Node.js or browsers are missing.**

Common Terminal Execution Commands (The three essential terminal commands to execute Playwright tests.)

```
# Run all tests headlessly
npx playwright test

# Run a specific test file
npx playwright test tests/first-test.spec.js

# Run tests with visible browser window
npx playwright test --headed
```

Let us examine your project's configuration file and review the top troubleshooting scenarios you might encounter.

The Purpose of playwright.config.js
The `playwright.config.js` file controls global settings for your entire test suite.

Here is a minimal, clean configuration example:

```javascript
// playwright.config.js
const { defineConfig } = require('@playwright/test');

module.exports = defineConfig({
  testDir: './tests',
  timeout: 30000,
  use: {
    headless: true
  }
});
```

• **`defineConfig()`:** A helper function that validates configuration options and provides autocomplete in VS Code.
• **`testDir`:** Specifies the folder where Playwright looks for test files (`./tests`).
• **`timeout: 30000`:** Maximum time in milliseconds (30 seconds) a single test can run before being terminated.
• **`use: { headless: true }`:** Controls default browser visibility. Set to `false` if you always want headed browsers during local debugging.

(Note: Your generated config file may have extra options for multiple browsers and reporting. Modify settings carefully rather than deleting the file).

Top 8 Troubleshooting Scenarios & Safe Diagnostics

1. `node` or `npm` command not found
• **Likely cause: Node.js is not installed or not added to your system PATH variable.**
• **Safe diagnostic: Open a new terminal window. Run `node -v`. If unrecognized, download and install Node.js LTS from nodejs.org, then restart your terminal.**

2. Node.js version is unsupported
• **Likely cause: You have an old Node version (e.g. Node 14 or 16).**
• **Safe diagnostic: Run `node -v`. If your version is below the Playwright minimum requirement, update Node.js to the current LTS release.**

3. `@playwright/test` package is missing
• **Likely cause: Dependencies were not installed or `node_modules` is missing.**
• **Safe diagnostic: Run `npm install` inside your project folder to restore all packages listed in `package.json`.**

4. Browser executable is missing
• **Error message: `Executable doesn't exist at /Users/.../Library/Caches/ms-playwright/...`**
• **Safe diagnostic: Run `npx playwright install` to download the required browser binaries.**

5. The test file is not discovered
• **Likely cause: The file is located outside the `tests/` folder or does not end in `.spec.js`.**
• **Safe diagnostic: Verify that your file is inside the `tests` directory and ends with `.spec.js` (e.g. `tests/first-test.spec.js`).**

6. Incompatible import vs require syntax
• **Error message: `Cannot use import statement outside a module` OR `require is not defined`.**
• **Safe diagnostic: If your `package.json` has `"type": "module"`, use `import { test } from '@playwright/test'`. If not, use `const { test } = require('@playwright/test')`. Never mix them.**

7. Public example website cannot be reached
• **Error message: `net::ERR_NAME_NOT_RESOLVED` or `net::ERR_CONNECTION_TIMED_OUT`.**
• **Safe diagnostic: Open your regular web browser and check if `https://example.com` loads. Check your computer's internet connection or proxy settings before changing test code.**

8. Assertion fails because content differs
• **Error message: `Timed out 5000ms waiting for expect(locator).toBeVisible()`.**
• **Safe diagnostic: Inspect the exact text and element role on the target page. Make sure the heading text matches exactly (remember JavaScript is case-sensitive).**

Checking Installed Playwright Version (Terminal command to verify the installed version of Playwright Test.)

```
npx playwright --version
```

Expected Output: Version 1.48.0

Node.js and npm, configure playwright.config.js, write your first pure JavaScript test navigating to https://example.com, verify heading visibility using web-first assertions, and execute the test in both headless and headed modes.

Task: Submit your first-test.spec.js code, terminal execution log, and written explanation meeting all 10 rubric criteria.

Assignment Starter Code (tests/first-test.spec.js)

```
// ============================================================================
// tests/first-test.spec.js - Your First Playwright Test
// ============================================================================

const { test, expect } = require('@playwright/test');

test('homepage displays the expected heading', async ({ page }) => {
  // 1. Navigate to the public practice website
  await page.goto('https://example.com');

  // 2. Locate the main heading and verify it is physically visible
  await expect(
    page.getByRole('heading', { name: 'Example Domain' })
  ).toBeVisible();
});
```

10 Rubric Requirements for Day 3 Assignment:

1. Project setup confirmation (playwright-js-practice initialized locally with package.json)
2. Import test and expect from @playwright/test (CommonJS require or ES Module import)
3. Test declaration with async ({ page }) fixture: test("...", async ({ page }) => { ... })
4. Asynchronous page navigation: await page.goto("https://example.com")
5. Accessible element locator: page.getByRole("heading", { name: "Example Domain" })
6. Web-first visibility assertion: await expect(locator).toBeVisible()
7. Terminal output showing successful headless execution (e.g. 1 passed)
8. Headed execution demonstrated or noted (--headed flag)
9. Written explanation defining the roles of test, expect, page, goto, and await
10. Diagnostic troubleshooting approach described for investigating test failures before assertions

Q1. What is Node.js, and why is it used in Playwright projects?

[Difficulty: Beginner] What Interviewer Evaluates: Understanding the execution environment and runtime architecture of modern automation tools.

Suggested Answer: Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside of a web browser on your computer. Playwright projects require Node.js because automated test scripts run on your operating system to launch and manage browser processes, communicate over WebSocket connections, read configuration files, and interact with the terminal.

Follow-Up Question: "How does JavaScript running in Node.js differ from JavaScript running inside Google Chrome?"

Q2. What is npm?

[Difficulty: Beginner] What Interviewer Evaluates: Package management fundamentals and dependency handling in QA codebases.

Suggested Answer: npm stands for Node Package Manager. It is the default package manager for Node.js and the largest software registry in the world. In automated testing, we use npm to install and update testing libraries (like @playwright/test), manage project dependencies, and execute test scripts defined in package.json.

Follow-Up Question: "What is the difference between npm and npx?"

Q3. What is the purpose of package.json?

[Difficulty: Beginner] What Interviewer Evaluates: Project structure knowledge and dependency tracking.

Suggested Answer: package.json is the core manifest file of a Node.js project. It stores project metadata (such as name and version), defines reusable terminal shortcut scripts (like "test": "playwright test"), and lists the exact packages and versions required to run the project under dependencies and devDependencies.

Follow-Up Question: "Why should package.json be committed to Git while node_modules is ignored?"

Q4. What is Playwright Test?

[Difficulty: Beginner] What Interviewer Evaluates: Distinguishing Playwright the library from Playwright Test the test runner.

Suggested Answer: Playwright Test (@playwright/test) is Microsoft's complete, dedicated end-to-end test runner. It provides the test() declaration structure, web-first assertions with expect(), automatic browser isolation fixtures ({ page }, { context }), parallel execution across multiple workers, and built-in HTML reporting.

Follow-Up Question: "Can Playwright be used as a standalone automation library without Playwright Test?"

Q5. What is the purpose of a .spec.js file?

[Difficulty: Beginner] What Interviewer Evaluates: Test discovery conventions and organizational standards.

Suggested Answer: In automated testing, "spec" stands for specification. Naming files with the .spec.js (or .test.js) convention signals to Playwright Test that the file contains executable test suites. By default, the Playwright test runner scans the tests directory and automatically discovers and runs files matching this naming pattern.

Follow-Up Question: "Can you customize which file extensions Playwright looks for in playwright.config.js?"

Q6. What does page.goto() do?

[Difficulty: Beginner] What Interviewer Evaluates: Browser navigation mechanics and asynchronous handling.

Suggested Answer: page.goto(url) is an asynchronous Playwright method that instructs the browser page to navigate to a target web address. It returns a Promise and automatically waits for the web page to fire the load event before resolving, ensuring your test does not proceed before the document is ready.

Follow-Up Question: "What navigation wait states can you configure in page.goto (such as domcontentloaded or networkidle)?"

Q7. What is the purpose of expect()?

[Difficulty: Beginner] What Interviewer Evaluates: Test assertions and web-first verification strategies.

Suggested Answer: expect() is Playwright's assertion function used to compare actual application state with expected outcomes. Playwright provides "web-first" assertions (such as expect(locator).toBeVisible() and expect(page).toHaveTitle()) that automatically poll and wait for the condition to be satisfied before timing out, eliminating flaky tests caused by slow rendering.

Follow-Up Question: "How do Playwright web-first assertions differ from generic Jest or Chai assertions?"

Q8. Why do Playwright tests frequently use async and await?

[Difficulty: Beginner] What Interviewer Evaluates: Asynchronous JavaScript foundations and test execution order.

Suggested Answer: Browser automation actions—such as opening a page, clicking an element, or fetching text—take time because they communicate across WebSocket connections to the browser and wait for rendering pipelines. Because these Playwright methods return Promises, using await pauses execution until each action physically finishes, preventing race conditions and ensuring steps execute in strict sequence.

Follow-Up Question: "What happens if you omit the await keyword before page.goto()?"

Q9. What is the difference between headless and headed execution?

[Difficulty: Beginner] What Interviewer Evaluates: Execution modes, performance considerations, and CI/CD pipelines.

Suggested Answer: In headless mode, browsers run silently in system memory without rendering a physical graphical user interface on your screen. This is significantly faster, uses fewer system resources, and is the standard mode for CI/CD pipelines. In headed mode (--headed), a visible browser window opens on your desktop, which is ideal for visually debugging clicks, form entries, and animations locally.

Follow-Up Question: "Why might a test pass in headed mode locally but fail in headless mode on a CI server?"

Q10. What would you investigate if a Playwright test fails before reaching the assertion?

[Difficulty: Beginner] What Interviewer Evaluates: Systematic debugging and root cause analysis skills.

Suggested Answer: If a test fails before reaching an assertion, the failure is usually caused by navigation, environment, or syntax issues rather than an application defect. I would check: 1) Syntax errors or missing await statements in the script, 2) Whether the target URL is accessible or returning a 404/500 network error, 3) Whether browser binaries are installed, 4) Whether a previous action failed (like an element not interactable or obscured by a modal), and 5) Inspect terminal stack traces and Playwright Trace Viewer for exact error details.

Follow-Up Question: "How does Playwright Trace Viewer help diagnose failures that occur before assertions?"

1. What is Node.js in the context of Playwright test automation?

Key Concept: Node.js is the runtime environment that executes JavaScript code outside the browser, allowing test scripts to control browser processes and files.

2. Which terminal command initializes a new Playwright project with the interactive setup wizard?

Key Concept: npm init playwright@latest launches the official interactive CLI setup wizard to configure your Playwright project.

3. When asked by the setup wizard whether to use TypeScript or JavaScript for this course, which should you choose?

Key Concept: This course uses pure JavaScript (.js) for all test scripts and exercises.

4. What is the purpose of the tests/ folder in a Playwright project?

Key Concept: By default, Playwright searches the tests/ directory for test files ending in .spec.js or .test.js.

5. In the test signature test("user login", async ({ page }) => { ... }), what is { page }?

Key Concept: { page } is an isolated browser page fixture provided automatically to each test by Playwright.

6. Which method instructs the browser page to navigate to a target web address?

Key Concept: page.goto(url) is the asynchronous method used in Playwright to navigate to a web address.

7. What does the assertion await expect(locator).toBeVisible(); do?

Key Concept: toBeVisible() is a web-first assertion that automatically polls and waits for the element to become visible, eliminating timing flakiness.

8. How do you run your Playwright test suite so that the browser window visibly opens on your monitor?

Key Concept: The --headed flag opens a visible browser window, allowing you to watch the test run in real time.

9. Why should the node_modules folder be excluded from Git version control?

Key Concept: node_modules contains downloaded dependencies and is huge. It is excluded via .gitignore and recreated on demand using npm install.

10. If running a test outputs "Executable doesn't exist at .../ms-playwright/...", what should you do?

Key Concept: Running npx playwright install downloads the missing browser binaries safely without touching your project files.